

UNITAT 2 · SETMANA 2

Crides al sistema i E/S de baix nivell

 Setmana 2 · dates per confirmar
  Laboratori TD1 (resolt + repte IA)
  4 animacions

LABORATORI TD1 · RESOLT

Llegir i escriure amb `read` i `write`

Els fonaments de la programació de sistemes: com entra un programa al nucli, com escriure i llegir en binari, i per què cal comprovar cada resultat.



Informe del laboratori

Entrega aquest laboratori amb un informe curt en LaTeX, amb la plantilla del curs: cada exercici explicat, el codi que importa i el que has executat al terminal. Pots fer servir IA generativa per escriure el LaTeX: declara-ho i verifica cada resultat. Plantilla, exemple i guia de Prism →

EXERCICIS

- | | | | |
|---|--|---|--|
| 1 | Com s'implementa una crida al sistema | 4 | <code>perror</code> és una crida al sistema? |
| 2 | <code>echo</code> amb <code>write</code> | 5 | Copiar amb <code>read</code> |
| 3 | Escriure un enter en binari | 6 | Tornar a llegir l'enter |
| | | 7 | Estructures en binari i farciment |

TD1 · 1

● Resolt · sortida real

Com s'implementa una crida al sistema

Enunciat. Una crida al sistema necessita una instrucció especial del processador (`syscall` a x86-64, `svc` a ARM, `ecall` a RISC-V). Desassembla la biblioteca de C amb `objdump -d` i troba com el programa diu al nucli *quina* crida vol. Després observa amb `strace` les crides que fa el teu `echo`.

Solució



Sortida real

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A4 -E '^[0-9a-f]+ <getppid@>'
0000000000ff710 <getppid@GLIBC_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

Sortida real

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A6 -E '^[0-9a-f]+ <__write_nocancel@>'
0000000000129060 <__write_nocancel@GLIBC_PRIVATE>:
    129060:    endbr64
    129064:    mov     $0x1,%eax
    129069:    syscall
    12906b:    cmp     $0xffffffffffffffff,%rax
    129071:    ja      129078 <__write_nocancel@GLIBC_PRIVATE+0x18>
    129073:    ret
```

El número de la crida va al registre `eax` (110 = `getppid`, 1 = `write` a x86-64) i els arguments a `rdi`, `rsi`, `rdx` ... La instrucció `syscall` canvia la CPU a mode nucli i salta a l'entrada del nucli, que fa servir `eax` com a índex a la seva taula de crides al sistema. En tornar, el resultat és a `rax`: un valor entre -4095 i -1 indica un error, i l'embolcall de glibc el converteix en `-1 + errno` (és el que fa `cmp $0xffffffffffffffff,%rax`).

Sortida real

```
$ strace -e trace=write ./echo hola mundo > /dev/null
write(1, "hola", 4)                = 4
write(1, " ", 1)                  = 1
write(1, "mundo", 5)               = 5
write(1, "\n", 1)                  = 1
+++ exited with 0 +++
```

Quatre crides `write`: una per paraula, una per a l'espai i una per al salt de línia. Cadascuna és una anada i tornada al nucli; a la unitat 4 veuràs com la biblioteca estàndard agrupa les escriptures en una memòria intermèdia per estalviar-les.

TD1 · 2

● **Resolt · provat**

`echo` amb `write(2)`

Enunciat. Escriu un programa `echo` que copii els seus arguments a la sortida estàndard fent servir només `write(2)`. Ha de comprovar els errors de `write` amb `exit_if`. Prova'l amb `./echo blabla < . 1>&0`.

Solució



C tds/td1/echo.c

```

/*
 * TD1 - exercise 3: echo with write(2).
 * Copies its arguments to standard output, separated by spaces, followed by
 a newline.
 * Try the error path with: ./echo blabla < . 1>&0 (stdout becomes a
 directory: write fails)
 */
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    for (int i = 1; i < argc; i++) {
        size_t len = strlen(argv[i]);
        ssize_t rc = write(STDOUT_FILENO, argv[i], len);
        exit_if(rc == -1, "write");
        /* write may write less than asked (pipe or socket full): it is not
 an error,
        but this program must then say so. write_all() in libutils loops
 instead. */
        exit_if((size_t)rc != len, "write: partial write");

        if (i < argc - 1)
            exit_if(write(STDOUT_FILENO, " ", 1) == -1, "write space");
    }
    exit_if(write(STDOUT_FILENO, "\n", 1) == -1, "write newline");
    return EXIT_SUCCESS;
}

```

Sortida real

```

$ ./echo hola mundo
hola mundo

```

Sortida real

```
$ ./echo blabla < . 1>&0
write: Bad file descriptor
[exit status 1]
```

`< .` obre el directori actual com a entrada estàndard (descriptor 0) i `1>&0` fa que la sortida (1) sigui una còpia del descriptor 0. Un directori obert només per llegir no es pot escriure: `write` retorna `-1` amb `errno = EBADF`, i `exit_if` ho informa.

TD1 · 3

● Result · provat

Escriure un enter en binari

Enunciat. Escriu `write_number`: escriu a la sortida estàndard el contingut de memòria d'un `long` igual a 5 i després el nombre de bytes escrits per la primera crida. Què veus al terminal? Mira la sortida amb `od`.

Solució



C tds/td1/write_number.c

```
/*
 * TD1 - exercise 4: write the memory content of a long, then the number of
 * bytes written.
 * The output is binary, not text: look at it with ./write_number | od -A d
 * -t x1
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    long value = 5;
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);    /* 8 bytes on
x86-64 */
    exit_if(written == -1, "write value");

    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00
00000016
```

Al terminal no surt res llegible: els bytes `05 00 00 00 00 00 00 00` no són caràcters imprimibles. `od` mostra la veritat: 8 bytes del `long` en ordre *little-endian* (el byte menys significatiu primer, com des a els enters un processador x86-64) i 8 bytes que valen 8, el que va retornar el primer `write`. `write` no converteix res en text: copia bytes.

TD1 · 4

● Resolt

perror és una crida al sistema?

Resposta

No. `perror` és a la secció 3 del manual: és una funció de la biblioteca de C que llegeix `errno`, busca el text de l'error i l'escriu a l'error estàndard... fent servir, internament, la crida al sistema `write`. Ho pots comprovar amb `strace ./echo blabla < . 1>&0`: hi apareix un `write(2, "write: Bad file descriptor\n", ...)`.

TD1 · 5

● Resolt · provat

Copiar l'entrada a la sortida amb `read(2)`

Enunciat. Escriu `read`: llegeix l'entrada estàndard amb `read` en un vector de mida `BUFFER_SIZE` i l'escriu a la sortida amb `write`, fins que es tanca l'entrada (`Ctrl + D`). Prova'l com a `cat`, copiant un fitxer i amb un directori com a entrada.

Solució

C tds/td1/read.c

```

/*
 * TD1 - exercise 6: copy standard input to standard output with
 * read(2)/write(2).
 * Stops at end of file (Ctrl-D in a terminal). Usages:
 * ./read          ./read < file          ./read < in > out
 * ./read < . (error)
 */
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#ifndef BUFFER_SIZE
#define BUFFER_SIZE 4096
#endif

int main(void)
{
    char buffer[BUFFER_SIZE];

    for (;;) {
        ssize_t n = read(STDIN_FILENO, buffer, BUFFER_SIZE);
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "read");
        if (n == 0) /* end of file: the writer closed its
side */
            return EXIT_SUCCESS;

        /* write can be partial: loop until the n bytes are out (see
write_all in libutils). */
        ssize_t done = 0;
        while (done < n) {
            ssize_t rc = write(STDOUT_FILENO, buffer + done, (size_t)(n -
done));
            exit_if(rc == -1, "write");
            done += rc;
        }
    }
}

```

Sortida real

```
$ ./read < input.txt
first line
second line
```

Sortida real

```
$ ./read < binary.bin > copy.bin && cmp binary.bin copy.bin && echo identical
identical
```

Sortida real

```
$ ./read < .
read: Is a directory
[exit status 1]
```



Dos detalls que separen un programa correcte d'un que funciona «gairebé sempre»

- `read` retorna **0** al final del fitxer, **-1** en cas d'error i, si no, *fins a* `BUFFER_SIZE` bytes: en pot retornar menys (una canonada, un terminal o un senyal escurcen la lectura).
- `write` també pot escriure menys del demanat: per això el bucle intern es repeteix fins que surten els `n` bytes. Si un senyal l'interromp abans d'escriure res, falla amb `EINTR` i n'hi ha prou de tornar-ho a provar.

TD1 · 6

● Result · provat

Llegir l'enter de `write_number`

Enunciat. Escriu un programa que llegeixi el que produeix `write_number` i ho imprimeixi amb `printf`.

Solució



C tds/td1/read_number.c

```

/*
 * TD1 - exercise 7: read what write_number produces and print it as text.
 * ./write_number | ./read_number      →  value = 5, bytes written = 8
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

/* read() may return fewer bytes than asked (e.g. from a pipe): loop until
`len` bytes. */
static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    long value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("value = %ld, bytes written = %zd\n", value, written);
    return EXIT_SUCCESS;
}

```

Sortida real

```

$ ./write_number | ./read_number
value = 5, bytes written = 8

```

`read_exact` repeteix `read` fins a obtenir exactament els bytes demanats: per una canonada els 16 bytes podrien arribar en dos trossos. Funciona perquè tots dos programes s'executen a la mateixa màquina: comparteixen la mida de `long` i l'ordre dels bytes. Entre màquines diferents caldria acordar un format (per exemple l'ordre de xarxa amb `htonl`, unitat 8).

TD1 · 7

● Resolt · provat

Estructures en binari i farciment

Enunciat. Converteix els programes anteriors en `write_struct` i `read_struct` per a una `struct nopad { char c1; long l; char c2; }` declarada a `utils.h`, amb `'a'`, 5 i `'b'`. Quina mida té l'estructura? Canvia amb `-03`? Com es pot fer més petita?

Solució

```
C tds/td1/write_struct.c

/*
 * TD1 - exercise 8: write a structure in binary. struct nopad is declared in
 * utils.h
 * so that write_struct and read_struct agree on its layout.
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    struct nopad value = { .c1 = 'a', .l = 5, .c2 = 'b' };
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);
    exit_if(written == -1, "write value");
    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

C tds/td1/read_struct.c

```

/*
 * TD1 - exercise 8: read the structure written by write_struct.
 * ./write_struct | ./read_struct      → { c1 = 'a', l = 5, c2 = 'b' },
bytes written = 24
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    struct nopad value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("{ c1 = '%c', l = %ld, c2 = '%c' }, bytes written = %zd\n",
           value.c1, value.l, value.c2, written);
    return EXIT_SUCCESS;
}

```

Sortida real

```

$ ./write_struct | ./read_struct
{ c1 = 'a', l = 5, c2 = 'b' }, bytes written = 24

```

C tds/td1/struct_sizes.c

```

/*
 * TD1 - exercise 8 (answers): size and padding of struct nopad, and two ways
 * to shrink it.
 * The compiler aligns `long` on 8 bytes: 7 padding bytes after c1 and 7
 * after c2 (arrays of
 * the structure must keep every `l` aligned). -O3 does not change the
 * layout: it is part of
 * the ABI, not an optimisation.
 */
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

struct reordered {          /* biggest member first: only 6 padding bytes at
the end */
    long l;
    char c1;
    char c2;
};

struct __attribute__((packed)) packed { /* no padding at all (slower,
unaligned access) */
    char c1;
    long l;
    char c2;
};

int main(void)
{
    printf("struct nopad      : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct nopad),
        offsetof(struct nopad, c1), offsetof(struct nopad, l),
offsetof(struct nopad, c2));
    printf("struct reordered : size %2zu  offsets l=%zu c1=%zu c2=%zu\n",
sizeof(struct reordered),
        offsetof(struct reordered, l), offsetof(struct reordered, c1),
offsetof(struct reordered, c2));
    printf("struct packed     : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct packed),
        offsetof(struct packed, c1), offsetof(struct packed, l),
offsetof(struct packed, c2));
}

```

```
return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./struct_sizes
struct nopad      : size 24  offsets c1=0 l=8 c2=16
struct reordered  : size 16  offsets l=0 c1=8 c2=9
struct packed     : size 10  offsets c1=0 l=1 c2=9
```

L'estructura ocupa **24 bytes** tot i que els seus camps sumen 10: el compilador alinea `long` a 8 bytes i afegeix 7 bytes de farciment després de `c1` i 7 al final (perquè, en un vector d'estructures, cada `l` quedi alineat). `-03` no canvia res: la disposició a la memòria forma part de l'ABI, no és una optimització. Per reduir-la: ordenar els camps de més gran a més petit (16 bytes) o empaquetar-la amb `__attribute__((packed))` (10 bytes, amb accessos no alineats més lents i no portables).



Corregit respecte de la versió heretada

`read_struct.c` no compilava: l'estructura no era a `utils.h` i feia servir un camp `c` que no existeix.

Llista de comprovació

Repte amb IA generativa · Un `cat` que mesura les seves pròpies crides al sistema

Objectiu. Amplia `read.c` perquè rebí la mida de la memòria intermèdia com a argument, mesura amb `strace -c` i amb el temps d'execució quantes crides `read` i `write` fa per a cada mida, i explica la corba.

Prompt inicial. Fes-lo servir a l'eina que prefereixis (Claude, ChatGPT, Gemini, Copilot, Codex...) amb la versió *resolta* del laboratori com a context. Demana primer un pla i revisa cada pas abans d'acceptar-lo.

Prompt

Tinc aquest programa en C que copia l'entrada estàndard a la sortida estàndard amb `read/write` i una memòria intermèdia de mida fixa [enganxa `read.c`]. Ajuda'm a: (1) rebre la mida de la memòria intermèdia com a argument (d'1 byte a 1 MiB) amb `malloc`; (2) escriure un script de bash que, per a les mides 1, 16, 256, 4096, 65536 i 1048576, copiï un fitxer de 50 MB, mesuri el temps amb `/usr/bin/time -f %e` i compti les crides amb `strace -c`; (3) explicar per què el temps baixa tant al principi i després s'estabilitza. No em donis la conclusió abans que vegi els números.

Com comprovar el resultat

- El fitxer copiat és idèntic (`cmp`) per a totes les mides.
- El nombre de crides `read` és $\approx$ mida del fitxer / mida de la memòria intermèdia (més una lectura final que retorna 0).
- Saps explicar amb les teves paraules el cost fix de cada crida al sistema (canvi de mode).

Declara l'ús de la IA: eina, prompts rellevants, què has comprovat tu i què has corregit. El que compta és que puguis explicar i verificar cada decisió.